

Automated Model Selection for Multivariate Time Series Forecasting

Xiaoxuan Fan*
Huazhong University of Science and
Technology
School of Cyber Science and
Engineering
Wuhan, China
fxxdx@hust.edu.cn

Qiankun Zhang
Huazhong University of Science and
Technology
School of Cyber Science and
Engineering
Wuhan, China
Ministry of Education of China
Key Laboratory of Cyberspace
Security
Zhengzhou, China
qiankun@hust.edu.cn

Jiaqi Sun*
China Nuclear Power Operation
Technology Co., Ltd.
Cyber Security Center
Wuhan, China
sunjq02@cnnp.com

Wei Xiang
Central China Normal University
Faculty of Artificial Intelligence in
Education
Wuhan, China
xiangwei@ccnu.edu.cn

Lingzhi Yi
Zhongnan University of Economics
and Law
School of Information Engineering
Wuhan, China
lzyi@zuel.edu.cn

Xianjun Deng†
Huazhong University of Science and
Technology
School of Cyber Science and
Engineering
Wuhan, China
dengxj615@hust.edu.cn

Shenghao Liu
Huazhong University of Science and
Technology
School of Cyber Science and
Engineering
Wuhan, China
liushenghao@hust.edu.cn

Abstract

Accurate multivariate time series forecasting (MTSF) is critical for intelligent web services in Web of Things. When confronted with unseen multivariate time series (MTS), the industry typically invests significant time and resources in training multiple models to identify the optimal model for deployment. This paper proposes a novel, efficient, and scalable MTSF model selection method that directly selects suitable MTSF methods based on data characteristics without extensive model training. Model selection is a core component of AutoML, which has made significant progress in recent years. However, existing methods incur high operational costs and cannot be directly applied to MTSF tasks. Moreover, there is a lack of a comprehensive and cohesive public time series library for MTSF model selection. To address these challenges, we compile the first large heterogeneous labeled MTSF model selection dataset,

called the ModelPile, which covers 41 mainstream datasets across 11 domains. We then propose AutoMTSF, a large model-enabled model selection method that transforms the MTSF model selection problem into a time series classification problem and utilizes the ModelPile to unlock large-scale multi-dataset training. AutoMTSF first uses the pre-trained large model to encode raw MTS. Given the coarse-grained limitations of large model encoding, Recursive Temporal Pattern Feature (RTPF) is proposed to capture both fine-grained and global temporal feature evolution, thereby effectively mapping data characteristics to the MTSF method space. Experiments comparing AutoMTSF with 2 baselines, 17 MTSF methods, and 4 large time series models show that AutoMTSF outperforms state-of-the-art methods while maintaining comparable execution time. This work represents a critical step in validating the accuracy and efficiency of large model-enabled classification for MTSF.

*co-first author

†corresponding author



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792332>

CCS Concepts

• Mathematics of computing → Multivariate statistics; • Information systems → Web services.

Keywords

Multivariate time series forecasting; Model selection; Web of Things

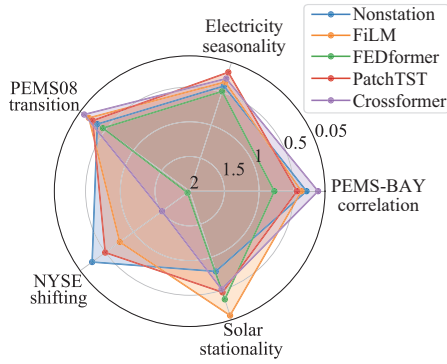


Figure 1: Mean absolute error (MAE) performance of five MTSF methods (Nonstation [28], FiLM [61], FEDformer [62], PatchTST [31], Crossformer [58]) on Electricity [41], PEMS08 [38], NYSE [13], Solar [23], and PEMS-BAY [25] datasets with distinct seasonality, transition, shifting, stationarity, and correlation characteristics.

ACM Reference Format:

Xiaoxuan Fan, Jiaqi Sun, Xianjun Deng, Qiankun Zhang, Wei Xiang, Shenghao Liu, and Lingzhi Yi. 2026. Automated Model Selection for Multivariate Time Series Forecasting. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792332>

1 Introduction

Web of Things (WoT), as an evolving ecosystem, relies on the ability to predict and adapt to changing patterns and user behavior [26]. Multivariate time series forecasting (MTSF) [21] plays a vital role in intelligent web service for WoT, which refers to the task of predicting future values of *multiple interdependent variables* based on historical observations. For instance, in the context of network traffic forecasting, MTSF may leverage past hourly measurements of throughput, resource block utilization, and user count to forecast their future values over upcoming hours. The MTSF problem is inherently challenging due to the complex and diverse characteristics of multivariate time series (MTS), such as seasonality and trends, and inter-variable dependencies.

There is a long history since the Vector Autoregression (VAR) [40] method to solve the MTSF problem. In subsequent years, machine learning-based methods, such as XGBoost [10] and Random Forest [42], have demonstrated superior performance in comparison. More recently, deep learning-based techniques [54] have emerged as the dominant paradigm. However, unfortunately, these models are often dataset/domain-specific. When presented with an unknown dataset, the industry must invest considerable time and resources in training and comparing multiple models to ascertain the most suitable one for deployment. This is a computationally intensive and resource-consuming process that is impractical for real-world applications.

This paper proposes an effective, efficient, and scalable solution that innovatively adopts a model selection method [9, 39] to solve heterogeneous MTSF. Model selection is a core module in Automated Machine Learning (AutoML), which systematically identifies

the optimal statistical model or algorithm from a set of candidates based on predefined performance metrics. In our task, our intention is to integrate the strengths of different models and select an appropriate MTSF method based on the characteristics of heterogeneous MTS without training a lot of models. However, applying existing AutoML-based model selection methods to MTSF can be non-trivial: (i) generating anomaly data and labels specifically for time series anomaly model selection and is not suitable for MTSF; (ii) difficulty in extracting predefined meta-features for each time series based on the data characteristics; (iii) requiring general objective optimization functions among candidate methods to search the optimal model configuration, which is time-consuming and requires repeated evaluations.

To address above challenges in MTSF model selection, our first contribution is *the first large-scale model selection dataset for MTSF*, named ModelPile. Public time series datasets are much smaller and task-specific compared to those in computer vision and natural language domains. Such datasets make it challenging for models to capture diverse data characteristics and facilitate model selection training. The ModelPile integrates 41 mainstream MTS datasets from 11 domains, containing 3,249,679 data samples. Specifically, we compile 41 datasets and preprocess these data using a rolling slice strategy, resulting in 341,734 subsequences. Building on ModelPile, we employ the TFB benchmark to uniformly evaluate the performance of 17 widely-used MTSF methods on these data. We identify the most suitable MTSF method and label each subsequence based on a comprehensive evaluation.

Our main technical contribution is *a novel viewpoint that performs feature encoding on raw MTS and transforms the MTSF model selection into a classification problem*. As an intuitive experiment, we evaluate five typical MTSF methods on a standard TFB benchmark [34] and five datasets. Figure 1 reveals that different methods win champions on different datasets exhibiting distinct temporal characteristics. Thus, the latent temporal characteristics of heterogeneous MTS map to suitable MTSF methods. Accordingly, we propose AutoMTSF, a large model-enabled MTSF model selection method that leverages the pre-trained large model MOMENT [16] to encode raw MTS and uses ModelPile for fine-tuning, aligning heterogeneous data features with the MTSF method category space. However, features encoded by MOMENT are coarse-grained and neglect local details, hindering classification accuracy. To address this, we propose *a novel Recursive Temporal Pattern Feature (RTPF) based on the most intrinsic characteristics of MTS data*. RTPF first extracts fine-grained static characteristics (seasonality, trend, correlation, and stationarity) within each sliding window, then concatenates these window-level features to further explore global dynamic evolution patterns. AutoMTSF integrates RTPF and MOMENT encoding, performs independent model selection for each time series subsequence, and offers two strategies—majority voting and mixed weighting—for final method determination.

Compared to traditional classification methods [12, 19], AutoMTSF demonstrates more balanced performance in time series classification due to its architecture and learning capabilities. In performance comparison with 2 model selection methods, 17 MTSF methods, and 4 large time series models [8, 16, 63], AutoMTSF achieves 1.88 times the average accuracy of MTSF methods and

4.44 times that of large models. The contributions are summarized as follows:

- We construct the first large-scale model selection dataset for MTSF, which consists of 41 mainstream multivariate time series datasets from 11 domains.
- We propose AutoMTSF, a large model-enabled model selection method, which transforms the challenging MTSF model selection problem into a time series classification problem. AutoMTSF effectively handles heterogeneous MTS with high accuracy, scalability, and flexibility.
- We propose Recursive Temporal Pattern Feature (RTPF) that can extract discriminative temporal pattern evolution and improve the accuracy of time series classification.
- Experiments measure the forecasting and computational performance of AutoMTSF, 2 baselines, 4 large time series models, and 17 MTSF methods. Results validate the superiority of AutoMTSF over state-of-the-art methods.

2 Related Work

Multivariate time series forecasting. Initially, statistical methods such as ARIMA [6] and VAR [40] were used for MTSF. Then, machine learning methods emerged, including well-known models like Random Forest [42], and GBRT [14]. These methods heavily rely on manual feature engineering and model design. Deep learning methods have gained widespread adoption recently. TCN [2] and DeepAR [37] utilize Convolutional Neural Networks (CNN)/Recurrent Neural Network (RNN) to capture time series dependencies. STGNNs [53] and MTGNNs [51] use graphs to represent inter-series relationships and use graph neural networks to predict traffic data. Transformer architecture have demonstrated efficacy in time series analysis. Informer [60], FEDformer [62], PatchTST [31], Triformer [11], and Autoformer [50] can capture complex temporal dynamics and dependencies. In contrast to common beliefs, MLP-based methods like NLinear [55] and DLinear [55], despite their simple structures and fewer parameters, have demonstrated strong forecasting accuracy. Recent benchmarks [34, 56] systematically evaluated MTSF methods and revealed that these methods typically perform well on data with certain temporal characteristics. No single MTSF method universally outperforms others.

Time series model selection. Current time series model selection primarily targets anomaly detection. METAOD [59] extracts meta-features from the target dataset and leverages historical experience to select the best model and hyperparameters. MOSPAT [9] is an end-to-end AutoML-based model and parameter selection method, which integrates a generative model to produce labeled data. Microsoft explored the efficacy of model selection for incoming data using 82 time-series statistical features [52].

3 Preliminaries and Problem Formulation

DEFINITION 1 (MULTIVARIATE TIME SERIES FORECASTING). Given a multivariate time series $X \in \mathbb{R}^{T \times N \times F}$ of T time points, where N is the number of variables and F is the feature dimension, MTSF aims to leverage the historical observed data with length L to predict the next future time points $X' \in \mathbb{R}^{H \times N \times F}$, where H is the forecasting horizon. Accordingly, we summarise MTSF as the following function:

$$\mathcal{F}(X^{t-L+1:t}) = X' \quad (1)$$

where $X^t \in \mathbb{R}^{N \times F}$ denotes all variable values at time t .

PROBLEM 1 (MTSF MODEL SELECTION). Given time series set $\mathcal{D}$ (the length T of time series $X \in \mathcal{D}$ is not fixed), candidate MTSF methods $\mathcal{B} = \{M_1, \dots, M_m\}$, and time step $t \in [L-1, T-H]$, we aim to establish a model selection method $\mathcal{M}$ that takes a time series X as input and returns a method $M \in \mathcal{B}$ ($\mathcal{M} : \mathcal{D} \rightarrow \mathcal{B}$), such that:

$$\mathcal{M}(X) = \arg \min_{M \in \mathcal{B}} \left\{ \text{Err} \left(M(X^{t-L+1:t}), X^{t+1:t+H} \right) \right\} \quad (2)$$

where the function $\text{Err}(x, y)$ calculates the error between predicted value x and true value y .

Accordingly, we transform **Problem 1** into a classification task that aligns time-series features with the MTSF method category space. The core task is to compute $\text{Err} \left(M(X^{t-L+1:t}), X^{t+1:t+H} \right)$ for all $X \in \mathcal{D}$, $M \in \mathcal{B}$, and use them to train model selection model.

4 Methodology

In this work, we construct a large-scale MTSF model selection dataset named ModelPile and propose AutoMTSF, a MTSF model selection method including large model encoding and RTPF feature.

4.1 ModelPile

We collate and preprocess 41 MTS datasets and construct the large-scale MTSF model selection dataset named ModelPile. The data samples in ModelPile span 11 domains and have a wide range of time-series characteristics such as stationarity, shifting, and frequency.

Divide MTS via rolling slice and forecasting strategy. To ensure the method can handle various datasets, we slice MTS into shifting subsequences of equal length. In Figure 2, green squares denote historical data (called lookback), orange squares denote forecast range (called horizon), and white squares denote the remaining MTS data. Each iteration generates a subsequence of length $s = H + L$, where H and L are horizon and lookback value. This design significantly reduces redundant information and accelerates training and inference. During rolling slice, historical data slides a fixed steps (called stride) in each iteration.

For dataset set $\mathcal{D}$, each MTS X with length T is divided into non-overlapping subsequences with equal length s . Let $\mathcal{D}_s$ denote the set of subsequences divided by X . When T is not evenly divided by s , the remaining part is added to the overlap between the first two windows, such that:

$$\mathcal{D}_s = \left\{ \begin{array}{l} \{X^{i*s,s} \mid \forall i \in [0, \lceil \frac{T}{s} \rceil]\} \\ \{X^{0,s}\} \cup \{X^{T-\lceil \frac{T}{s} \rceil+i*s,s} \mid \forall i \in [0, \lceil \frac{T}{s} \rceil]\} \end{array} \right\} \quad \begin{array}{l} \frac{T}{s} = \lceil \frac{T}{s} \rceil \\ \frac{T}{s} > \lceil \frac{T}{s} \rceil \end{array} \quad (3)$$

where $X^{i*s,s}$ represents the i th subsequence with length s . To facilitate the representation, we simplify to $X^{i,s}$.

Label each subsequence based on TFB benchmark. ModelPile contains 341,734 subsequences. For each subsequence, we use the TFB benchmark and perform 17 MTST methods to forecast the last H steps based on the previous L steps. Notably, only the initial L steps of the subsequence are provided to the MTSF method for prediction, thereby eliminating any potential for data leakage. Then, we calculate the average error metric. The method with the lowest MAE is selected as the label.

Minimize overfitting or accuracy illusions using careful train-test splitting. We use subsequences obtained by different

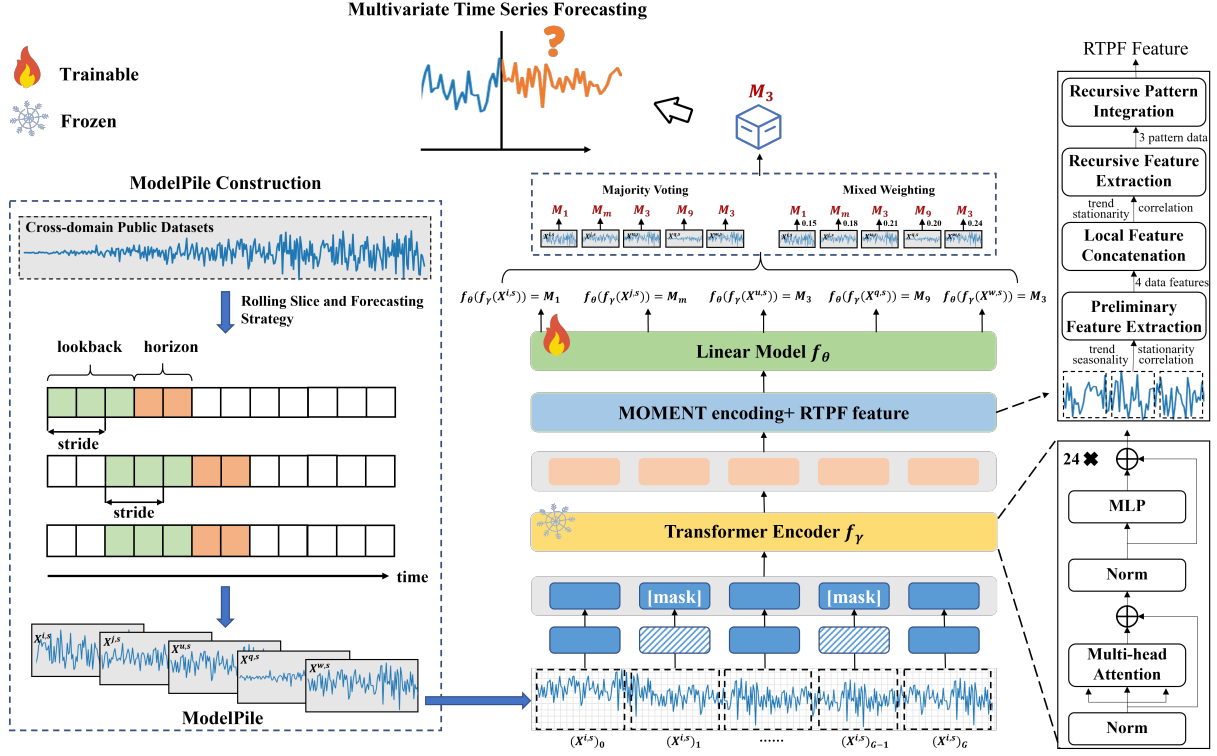


Figure 2: Overview of AutoMTSF. Left: Use Rolling slice and forecasting strategy to process large-scale cross-domain public datasets and construct ModelPile. Right: Use Transformer-based AutoMTSF to extract MOMENT encoding and RTPF feature, classify subsequences in ModelPile, and select MTSF method for the time series using majority voting or hybrid weighting.

slice strategies (different strides, same horizon) for training and testing. Common train-test splitting methods simply split each dataset into disjoint sets. However, subsequences from the same slice strategy share characteristics despite the data being disjoint, limiting the evaluation of MTSF model selection for heterogeneous MTS. To avoid overlap and maintain adequate sample size, we train and test model selection methods separately using subsequences from the same dataset, but with different strides (60 steps for training and 100 steps for testing).

4.2 AutoMTSF

As shown in Figure 2, AutoMTSF includes four steps: (i) Large model-enabled representation: encode each subsequence using pre-trained MOMENT; (ii) RTPF feature extraction: extract discriminative temporal feature; (iii) Fine-tuning and classification with ModelPile: classify each subsequence to obtain an MTSF method; (iv) Final model selection: select optimal MTSF method for MTS based on its subsequence results via model selection module.

4.2.1 Large model-enabled representation. We transform the MTSF model selection problem into a classification task, which also poses the challenge of which classifier is better. Thus, we use the time series fundamental model MOMENT [16] for data representation, and fine-tunes it using ModelPile.

Initially, the subsequence $X^{i,s} \in \mathbb{R}^{s \times N \times F}$ undergoes reversible instance normalization and is decomposed into G non-overlapping patches. A learnable mask randomly masks a few patches, and these processed patch embeddings are fed into the Transformer encoder. Finally, MOMENT reconstructs both masked and unmasked patches using transformed patch embeddings via a lightweight reconstruction head. Model parameters are updated by the loss:

$$\min_{\gamma, \beta} \mathcal{L}_M = \frac{1}{N_X} \sum_{i=1}^{N_X} \sum_{j=1}^G \| (X^{i,s})_j - f_\beta(f_\gamma((X^{i,s})_j)) \| \quad (4)$$

where γ and β are parameters of the MOMENT encoder and reconstruction head, f_γ and f_β are functions of MOMENT and reconstruction head, N_X is the total subsequence number of ModelPile, $(X^{i,s})_j$ is the j th patch of the subsequence $X^{i,s}$.

Our AutoMTSF utilizes the pre-trained MOMENT with a large-sized encoder as the representation module. The large MOMENT consists of a 24-layer modified Transformer architecture [35] with 16 attention heads, a hidden dimension of 1024, and a feed-forward neural network size of 4096, which generates approximately 385 million parameters. This model has been trained on 4 task-specific, widely-used public repositories. It can represent time series across different domains and characteristics.

4.2.2 RTPF Feature Extraction. The Moment encoding of subsequences is coarse-grained, which is insufficient for effective mapping to the candidate MTSF method category space. Accordingly, we propose a novel Recursive Temporal Pattern Feature (RTPF) to supplement this. RTPF initially extracts local static attributes of MTS at a fine-grained sliding window level, followed by concatenating window-level features for recursive feature extraction to capture global feature evolution patterns.

Preliminary Feature Extraction. Given the subsequence $X^{i,s}$, a sliding window of length P is applied to segment $X^{i,s}$ with stride $\lfloor P/2 \rfloor$. For the k^{th} windowed data, we compute seasonality, trend, stationarity, and correlation, respectively.

(1) Seasonality: refers to the phenomenon where variations in a time series recur at specific intervals. We compute the seasonality strength of a time series by analyzing its frequency domain characteristics through Fast Fourier Transform (FFT). Algorithm 1 in Appendix details the calculation process.

(2) Trend: refers to the long-term change or pattern that occurs over time. Intuitively, it represents the general direction of data variation. To calculate the trend strength, we first conduct a linear regression analysis on time series X to derive the correlation coefficient r for the fitted curve, and subsequently regard r^2 as the trend strength of X .

(3) Stationarity: refers to a time series where the mean and variance of any observation are constant, and the autocovariance depends solely on the time interval. Due to the scarcity of strictly stationarity, we focus on weak stationarity as in previous research [29, 34]. We use the Augmented Dick-Fuller (ADF_{Full}) test to calculate the stationarity score *Station* of time series X , such as:

$$ST(X) = 1 - \min(1.0, ADF_{Full}(X) * 10) \quad (5)$$

(4) Correlation: refers to the phenomenon where different variables in a multivariate time series may exhibit common trends or patterns. The correlation strength of multivariate time series X is calculated as follows.

$$Co(X) = \max\left(0, \min\left(1, \frac{2}{n(n-1)} \sum_{i=1}^{n-1} \sum_{j=i+1}^n |r_{ij}|\right)\right) \quad (6)$$

where r_{ij} denotes the Pearson correlation coefficient between the variables i and j . $\frac{2}{n(n-1)}$ is the reciprocal of the total number of unique correlation coefficients among all variables in X .

Accordingly, we construct seasonality, trend, stationary, and correlation feature data based on the seasonality, trend, stationarity, and correlation of all sliding windows, respectively.

Recursive Feature Extraction. Since four feature data are all extracted by sliding windows along the time, they can also be regarded as time series data. To observe the evolution of feature data over time, we separately extract the trend, stationarity, and correlations of the four feature data using the same method as before to construct trend pattern data P_{tr} , stationarity pattern data P_{st} , and correlation pattern data P_{co} .

Formally, given the seasonality feature data D_{se} , trend feature data D_{tr} , stationarity feature data D_{st} , and correlation feature data

D_{co} , P_{tr} , P_{st} , and P_{co} are defined as:

$$P_{tr} = \{Tr(D_{se}), Tr(D_{tr}), Tr(D_{st}), Tr(D_{co})\} \quad (7)$$

$$P_{st} = \{St(D_{se}), St(D_{tr}), St(D_{st}), St(D_{co})\} \quad (8)$$

$$P_{co} = \{Co(D_{se}), Co(D_{tr}), Co(D_{st}), Co(D_{co})\} \quad (9)$$

Recursive Pattern Integration. We concatenate P_{tr} , P_{st} , and P_{co} to construct RTPF. Given that excessively long features may be overly sparse and challenging to analyze, Principal Component Analysis (PCA) is used to reduce the feature dimensionality, resulting in the compact feature RTPF.

4.2.3 Fine-tuning and Classification with ModelPile. We concatenate RTPF and MOMENT encoding to serve as the ultimate input features for classification. We freeze the MOMENT and replace its reconstruction head with a simple linear model that maps patch embeddings to m logits. We fine-tune AutoMTSF with ModelPile to predict the most suitable MTSF method for each subsequence in a time series. That is, we need to learn a mapping $f : X^{i,s} \rightarrow \mathcal{B} = \{M_1, \dots, M_m\}$ from subsequences in the ModelPile to a finite set of classes. Formally, the classification process is defined as follows:

$$f_{\theta}(\text{concat}(f_Y(X^{i,s}), RTPF(X^{i,s}))) = M_i \quad (10)$$

where f_{θ} is the function of the added linear model, and θ is the parameter of the linear model we want to fine-tune.

Accordingly, the linear model is updated to select the MTSF method that matches the label, such that:

$$\min_{\theta} \mathcal{L}_f = -\frac{1}{N_X} \sum_{i=1}^{N_X} \sum_j^m c_{ij} \log p^j(X^{i,s}) \quad (11)$$

where $p^j(X^{i,s})$ denotes the probability that AutoMTSF selects the MTSF model M_j for subsequence $X^{i,s}$, c_{ij} represents the one-hot vector corresponding to the label M_j .

4.2.4 Final Model Selection. AutoMTSF cannot guarantee to select consistent MTSF methods for all subsequences of the same MTS. Therefore, this module offers both majority voting and mixed weighting schemes to determine the most suitable MTSF method. We compare the performance of the two schemes in Section 5.5.

In majority voting, given an AutoMTSF model $\mathcal{P}$ applied to subsequence set $\mathcal{D}_s$ of time series X , $\mathcal{P}(\mathcal{D}_s)$ represents the set of methods chosen for subsequence $X^{i,s} \in \mathcal{D}_s$. The model selection process via majority voting can be described as:

$$f(X, \mathcal{P}) = \arg \max_{M_i \in \mathcal{P}(\mathcal{D}_s)} \sum_{X^{i,s} \in \mathcal{D}_s} \mathbb{1}_{[\mathcal{P}(X^{i,s})=M_i]} \quad (12)$$

In mixed weighting, we combine time decay weighting with time series volatility weighting for model selection. Time decay weighting is based on the temporal position of subsequences, and results obtained from recent data are given higher weights. Formally, the time decay weight of the i th subsequence $X^{i,s}$ is $w_d^i = e^{\lambda(i-i_{min})}$, where λ is the decay factor and $i_{min} = 1$.

The time series volatility w_v^i of the subsequence $X^{i,s}$ is correlated with its standard deviation σ_i , which is calculated as:

$$w_v^i = \sigma_i + 0.1 \times \max(\sigma) \quad (13)$$

Table 1: Model selection accuracy(Acc) and Macro-F1 score (MF1) with different horizons.

Horizon	AutoMTSF		ConvNet		ResNet		InceptionTime	
	Acc	MF1	Acc	MF1	Acc	MF1	Acc	MF1
36	92.95	95.63	46.42	34.07	28.00	14.92	48.00	42.54
48	90.00	90.92	45.70	32.33	28.00	13.73	53.00	41.35
60	85.64	82.41	44.05	30.17	28.00	13.31	45.00	30.48
96	96.73	98.06	47.42	36.29	35.40	16.65	64.00	54.35

Therefore, the weight of the subsequence $X^{i,s}$ is:

$$w_i = \frac{w_d^i \times w_v^i}{\sum_{j=1}^{|X|} w_d^j \times w_v^j} \quad (14)$$

where $|X|$ is the total number of subsequences of X . Accordingly, model selection process by mixed weighting can be described as:

$$f(X, \mathcal{P}) = \arg \max_{M_i \in \mathcal{P}(\mathcal{D}_s)} \sum_{X^{i,s} \in \mathcal{D}_s \ \& \ \mathcal{P}(X^{i,s})=M_i} w_i \quad (15)$$

5 Experiments

5.1 Experiment Setup

Dataset. We use 41 mainstream MTS datasets to construct ModelPile, which cover 11 domains such as web, network, environment, and etc. The details of the datasets are shown in the Appendix B.

Candidate MTSF Methods. We select 17 widely-used MTSF methods as candidate MTSF methods for AutoMTSF: iTransformer [27], TimeXer [46], VAR [40], LinearRegression (Linear) [18], NLinear [55], DLinear [55], RNN [22], MICN [45], TimesNet [49], TCN [2], PatchTST [31], Crossformer [58], FEDformer [62], Non-stationary Transformer (Nonstation) [28], Informer [60], Tri-former [11], and FiLM [61]. Appendix C provides more details.

Model Selection Baseline. We compare AutoMTSF with two baselines, METAOD [59] and Auto-Selector [52]. METAOD and Auto-Selector are model selection methods for time series anomaly detection through meta-feature extraction and temporal statistical feature learning, respectively. For fairness, we adapted METAOD and Auto-Selector to the MTSF task for comparison.

Metric. We use MAE and Mean Squared Error (MSE) to evaluate forecasting performance.

Implementation details. We follow the parameter setting set by TFB when performing 17 MTSF methods to construct ModelPile. We uniformly split data using 7:3 when training AutoMTSF and baselines. A weighted cross-entropy loss is used with training epoch of 15. We run each method 3 times and report the average results. Unless otherwise specified, the AutoMTSF reported below utilizes majority voting in the model selection module.

5.2 Forecasting Accuracy Evaluation

We compare the forecasting performance of AutoMTSF with 2 baselines and 17 candidate MTSF methods. Figure 3 shows the overall MAE over 41 datasets (i.e., each box-plot corresponds to 213,608 accuracy values). AutoMTSF outperforms all baselines and existing MTSF methods ($p < 0.01$). We can conclude that AutoMTSF

greatly improves SOTA MTSF methods. This is because AutoMTSF uses MOMENT as the classifier and is fine-tuned on the ModelPile, which enables AutoMTSF to handle heterogeneous data. Besides, ModelPile labels the most suitable MTSF method based on data features, so AutoMTSF can combine the advantages of candidate MTSF methods to formulate the best scheme for diverse datasets.

5.3 Classification Accuracy Evaluation

Since AutoMTSF converts MTSF model selection into a classification problem, we compare AutoMTSF with three time series classifiers, ConvNet [47], ResNet [17], and InceptionTime [20]. They are proven to be outstanding performing classifier for model selection in [39]. Similar to AutoMTSF, ModelPile is utilized to train classifiers and the majority rule is applied to select the final MTSF method. We adopt a learning rate of $10e-5$ and the batch size of 256.

Results in Table 1 show that AutoMTSF’s classification accuracy and MF1 exceed the best-performing ConvNet by 45.43% and 58.54% on average across 41 datasets. AutoMTSF, built on MOMENT and RTPF, effectively represents heterogeneous MTS. Conventional classification methods, despite training on the ModelPile, still struggle to map data features to the 17 MTSF method categories, resulting in stochastic classification.

5.4 Transfer Performance Evaluation

For new incoming unknown data, AutoMTSF still performs well in model selection. To validate this, we conduct two transfer experiments on AutoMTSF. Initially, we randomly select 15-35 datasets from 41 datasets to train AutoMTSF, and then test it on the remaining datasets. The transfer performance with horizon=96 is shown in Figure 4. As the number of the training dataset increases, the MAE and MSE metrics decrease. Despite training on only 15 datasets, AutoMTSF demonstrates promising forecasting performance on the remaining 26 unseen datasets.

Furthermore, we train AutoMTSF using partial domain data from 11 domains across 41 datasets and tested it on the remaining unknown domains, as depicted in Figure 5. With knowledge of only half the domain data (unknown for web, electricity, energy, stock, and health), AutoMTSF achieves MAE and MSE values of 1.068 and 13.112, respectively. AutoMTSF’s performance is attributed to its pre-trained MOMENT encoding and RTPF feature extraction, which effectively extract MTS features and establish an effective mapping to the appropriate MTSF method space.

5.5 Majority Voting vs. Mixed Weighting

AutoMTSF offers majority voting and mixed weighting schemes to select the final method. The average forecasting performance of the two schemes with horizon $H = \{36, 48, 60, 96\}$ on 41 datasets across 11 domains is shown in Table 2. The forecasting performance of the two schemes is comparable, with the majority voting scheme exhibiting slightly lower forecasting error than the mixed weighting scheme across most datasets.

5.6 Comparison with Large Time Series Model

We also compare the forecasting performance of AutoMTSF with existing large time series models, including MOMENT (large) [16],

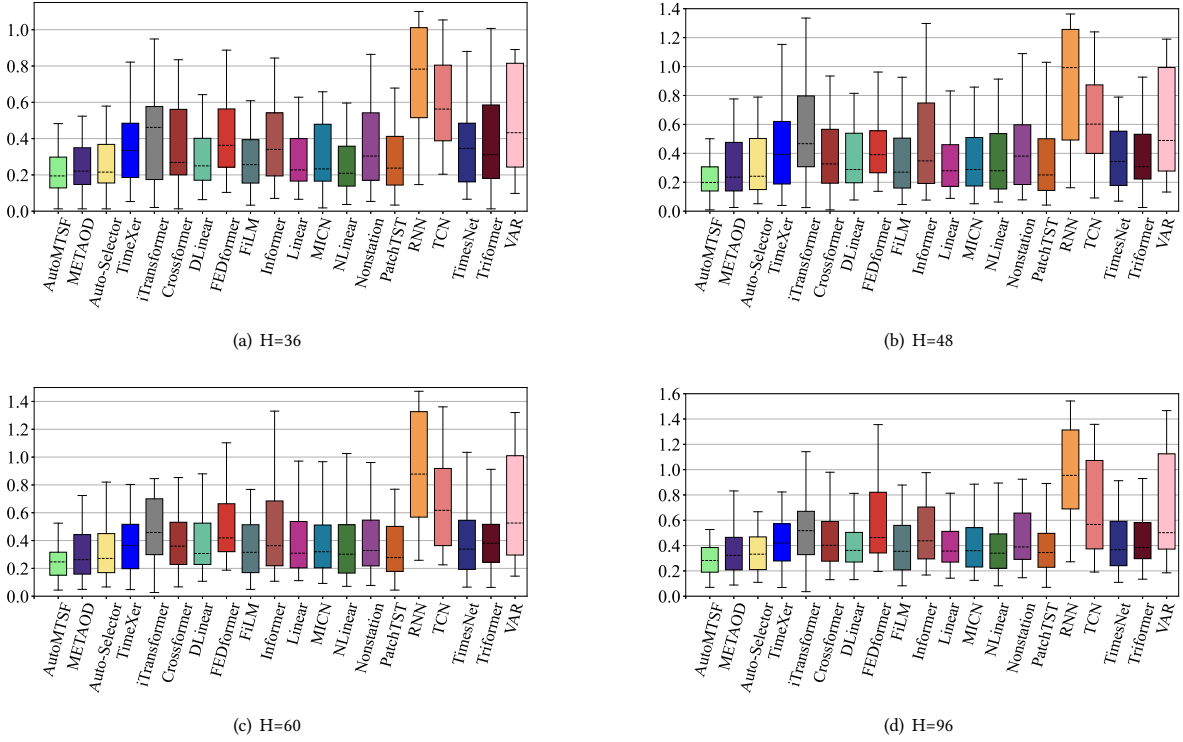


Figure 3: MAE performance of AutoMTSF, 2 baselines, and 17 candidate MTSF methods across 41 datasets with horizons $H = \{36, 48, 60, 96\}$. A lower MAE value indicates better forecasting performance.

Table 2: Forecasting results of AutoMTSF with majority voting/mixed weighting and large time series models on 41 datasets with horizons $H = \{36, 48, 60, 96\}$. A lower MAE/MSE indicates better performance and the best results are marked in bold.

Domain	Dataset number	AutoMTSF (majority)		AutoMTSF (mixed)		MOMENT_T		MOMENT		TEMPO		GPT4TS		Chronos-T5	
		MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE	MAE	MSE
Bank	1	0.498	0.517	0.495	0.517	0.627	0.736	0.879	1.279	1.160	2.154	0.939	1.353	0.666	0.690
Economic	2	0.732	12.010	0.732	12.010	1.644	16.343	1.760	18.485	2.253	18.932	1.570	19.010	2.074	20.745
Electricity	5	0.267	0.143	0.268	0.145	0.313	0.230	0.535	0.587	1.462	3.351	0.530	0.594	0.869	0.879
Energy	3	0.337	0.464	0.336	0.464	0.545	0.543	0.860	0.905	1.195	2.189	0.860	0.945	0.938	0.969
Environment	8	0.303	0.206	0.322	0.253	2.176	0.572	1.278	0.696	2.831	2.022	1.387	0.739	1.042	0.581
Health	2	0.653	1.516	0.662	1.531	2.131	2.688	2.974	2.200	2.509	2.405	2.264	3.316	0.725	1.530
Nature	2	0.187	0.105	0.192	0.107	0.292	0.265	0.677	0.851	1.256	2.817	0.675	0.848	0.404	0.541
Network	10	0.241	0.246	0.242	0.247	1.830	0.401	2.332	0.674	2.301	1.818	0.760	0.786	0.931	0.942
Stock	2	0.357	0.235	0.394	0.266	0.980	1.424	1.226	2.409	1.905	5.992	1.066	1.584	0.948	1.740
Transportation	5	0.185	0.110	0.187	0.112	0.350	0.435	0.758	1.065	1.526	3.565	0.792	1.168	1.556	1.713
Web	1	0.948	94.718	0.942	96.543	1.883	120.980	2.338	127.116	2.472	132.946	2.322	126.94	2.148	120.904
Avg		0.428	10.025	0.434	10.200	1.161	13.147	1.420	14.206	1.897	16.199	1.197	14.298	1.118	13.7499

MOMENT_T (fine-tuned), TEMPO (80M params) [8], GPT4TS (fine-tuned) [63], and Chronos-T5 (base) [1]. Table 2 reports average MAE and MSE metrics of AutoMTSF and large time series models on 11 domain with lookback $L = 336$ and horizon $H = \{36, 48, 60, 96\}$. AutoMTSF achieves a 63.14% reduction in MAE and a 23.75% reduction in MSE compared to the top-performing MOMENT_T across all domains. AutoMTSF has both heterogeneous data processing capabilities like large models and superior forecasting capabilities for domain-specific data like MTSF methods.

5.7 Inference Time Evaluation

We compare the inference time of AutoMTSF with candidate MTSF methods. The inference time for AutoMTSF includes the time required to select the MTSF method and the time required for the selected method to predict. The inference time for candidate MTSF methods include the time it takes for prediction. The experiments are conducted on a system equipped with the Intel(R) Xeon(R) Gold 6234 CPU @ 3.30GHz, three NVIDIA QUADRO RTX 5000 GPU, and

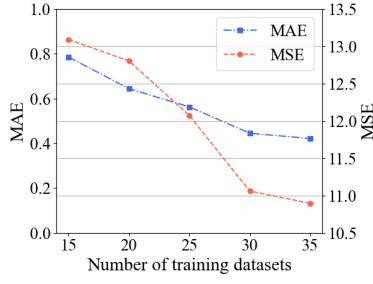


Figure 4: AutoMTSF performance with different number of known datasets.

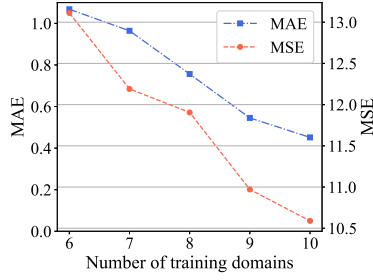


Figure 5: Transfer performance of AutoMTSF with different number of known domains.

running Ubuntu 18.04.6 LTS. Each method was configured with a batch size of 8. Due to space limitations, we chose 11 MTSF methods and three datasets of large (PEMS-BAY), medium (AQShunyi), and small (NASDAQ) size.

In Figure 6, the y-axis represents the inference time per sample in milliseconds and x-axis denotes the method's best MAE performance with horizon $H = 96$. We observe that AutoMTSF has outstanding forecasting accuracy while maintaining the same magnitude of inference time as other MTSF methods. Besides, inference time generally increases with dataset size.

5.8 Ablation Study

This subsection investigates the functionality of each module within AutoMTSF. AutoMTSF primarily comprises two modules: (i) feature extraction, encompassing MOMENT encoding and RTPF feature extraction; and (ii) model selection, offering majority voting and mixed weighting strategies. We sequentially remove these sub-modules to investigate their impact on AutoMTSF's model selection performance. As shown in Table 3, under majority and mixed strategies, the removal of MOMENT encoding resulted in average increases of 0.374, 2.852, and 0.420 in MAE, MSE, and RMSE, respectively. Similarly, the ablation of RTPF led to average increases of 0.668, 4.084, and 0.588 in MAE, MSE, and RMSE, respectively.

6 Conclusion

When faced with unseen datasets, the industry must expend significant time and resources training multiple models to determine the most suitable approach for deployment. To address this, this paper

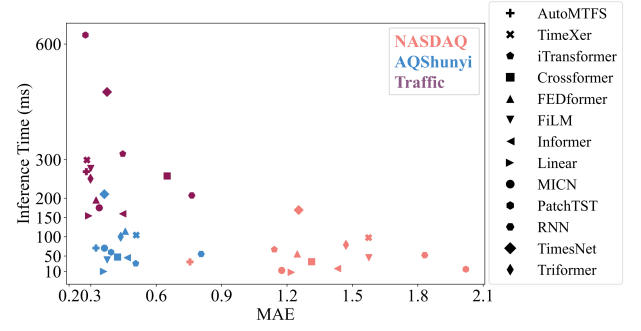


Figure 6: Comparison of inference time and MAE metrics for AutoMTSF and MTSF methods.

Table 3: Ablation study.

Method	MAE	MSE	RMSE
AutoMTSF(majority)	0.428	10.025	3.167
AutoMTSF(majority) wo MOMENT	0.797	12.831	3.582
AutoMTSF(majority) wo RTPF	1.088	14.130	3.759
AutoMTSF(mixed)	0.434	10.200	3.194
AutoMTSF(mixed) wo MOMENT	0.813	13.098	3.619
AutoMTSF(mixed) wo RTPF	1.109	14.263	3.777

develops an MTSF model selection method to select the suitable MTSF method for heterogeneous MTS based on its characteristics, without extensive model training. We first construct ModelPile, the first large-scale collection of MTS labelled with the optimal MTSF method. Then, we propose AutoMTSF, a large model-enabled MTSF model selection method that performs MOMENT encoding and RTPF feature extraction on raw data, then converts model selection problem into a time-series classification problem, and utilizes ModelPile for training. Comparisons with 2 baselines, 17 MTSF methods, and 4 SOTA large time-series models on 41 MTS datasets demonstrate that AutoMTSF achieves the best forecasting performance.

The current version of AutoMTSF can only select relatively suitable MTSF methods for heterogeneous MTS. Future efforts can further enhance model selection capabilities through tensor fusion of multiple features (such as RTPF and MOMENT encoding) or secondary classification.

Acknowledgment

This work was supported by National Key Research and Development Program of China (2022YFE0138600); National Natural Science Foundation of China (625B2078, U24B20153, 62272182, 62302183, 62472442); Fundamental Research Funds for the Central Universities, Huazhong University of Science and Technology (HUST) under Grant YCJJ20251306; Open Foundation of Key Laboratory of Cyberspace Security, Ministry of Education of China (Grant KLCS20240401); CCF-DiDi GAIA Collaborative Research Funds (Grant CCF-DiDi GAIA 202522), and Foundation of Key Laboratory of Intelligent Support Technology for Complex Environments, Ministry of Education (b1202502).

References

- [1] Abdul Fatir Ansari, Lorenzo Stella, Caner Turkmen, Xiyuan Zhang, Pedro Mercado, Huibin Shen, Oleksandr Shchur, Syama Syndar Rangapuram, Sebastian Pineda Arango, Shubham Kapoor, Jasper Zschiesner, Danielle C. Mad-dix, Michael W. Mahoney, Kari Torkkola, Andrew Gordon Wilson, Michael Bohlke-Schneider, and Yuyang Wang. 2024. Chronos: Learning the Language of Time Series. *Transactions on Machine Learning Research* (2024). <https://openreview.net/forum?id=gerNCVqqtR>
- [2] S. Bai, J. Z. Kolter, and V. Koltun. 2018. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. (2018). doi:10.48550/arxiv.1803.01271
- [3] Souhaib Ben Taieb, Gianluca Bontempi, Amir F. Atiya, and Antti Sorjamaa. 2012. A review and comparison of strategies for multi-step ahead time series forecasting based on the NN5 forecasting competition. *Expert Systems with Applications* 39, 8 (2012), 7067–7083. doi:10.1016/j.eswa.2012.01.039
- [4] Sathvik Bhaskarpandit. 2020. Wind Power Forecasting. <https://www.kaggle.com/datasets/theforcecoder/wind-power-forecasting>.
- [5] Peter Bodik, Wei Hong, Carlos Guestrin, Sam Madden, Mark Paskin, and Romain Thibaux. 2004. Intel Lab Data. <http://db.csail.mit.edu/labdata/labdata.html>.
- [6] G. E. P. Box and David A. Pierce. 1970. Distribution of Residual Autocorrelations in Autoregressive-Integrated Moving Average Time Series Models. *J. Amer. Statist. Assoc.* 65, 332 (1970), 1509–1526. doi:10.1080/01621459.1970.10481180
- [7] Norbert Budincsevity. 2016. Weather in Szeged 2006–2016. <https://www.kaggle.com/datasets/budincsevity/szeged-weather>.
- [8] Defu Cao, Furong Jia, Sercan O Arik, Tomas Pfister, Yixiang Zheng, Wen Ye, and Yan Liu. 2024. TEMPO: Prompt-based Generative Pre-trained Transformer for Time Series Forecasting. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=YH5w12OUuU>
- [9] Sourav Chatterjee, Rohan Bopadikar, Marius Guerard, Uttam Thakore, and Xiaodong Jiang. 2022. MOSPAT: AutoML based Model Selection and Parameter Tuning for Time Series Anomaly Detection. arXiv:2205.11755 [cs.LG]
- [10] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (San Francisco, California, USA) (KDD '16). Association for Computing Machinery, New York, NY, USA, 785–794.
- [11] Razvan-Gabriel Cirstea, Chenjuan Guo, Bin Yang, Tung Kieu, Xuanyi Dong, and Shirui Pan. 2022. Triformer: Triangular, Variable-Specific Attentions for Long Sequence Multivariate Time Series Forecasting. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-22*, Lud De Raedt (Ed.). International Joint Conferences on Artificial Intelligence Organization, 1994–2001. doi:10.24963/ijcai.2022/277 Main Track.
- [12] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xi-aohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *ICLR* (2021).
- [13] Fuli Feng, Xiangnan He, Xiang Wang, Cheng Luo, Yiqun Liu, and Tat-Seng Chua. 2019. Temporal Relational Ranking for Stock Prediction. *ACM Trans. Inf. Syst.* 37, 2, Article 27 (March 2019), 30 pages. doi:10.1145/3309547
- [14] Jerome H. Friedman. 2001. Greedy function approximation: A gradient boosting machine. *Annals of Statistics* 29 (2001), 1189–1232.
- [15] Jan Gasthaus, Konstantinos Benidis, Yuyang Wang, Syama Sundar Rangapuram, David Salinas, Valentin Flunkert, and Tim Januschowski. 2019. Probabilistic Forecasting with Spline Quantile Function RNNs. In *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics (Proceedings of Machine Learning Research, Vol. 89)*, Kamalika Chaudhuri and Masashi Sugiyama (Eds.). PMLR, 1901–1910. <https://proceedings.mlr.press/v89/gasthaus19a.html>
- [16] Mononito Goswami, Konrad Szafer, Arjun Choudhry, Yifu Cai, Shuo Li, and Artur Dubrawski. 2024. MOMENT: A Family of Open Time-series Foundation Models. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*, Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett, and Felix Berkenkamp (Eds.). PMLR, 16115–16152. <https://proceedings.mlr.press/v235/goswami24a.html>
- [17] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 770–778. doi:10.1109/CVPR.2016.90
- [18] Julien Herzen, Francesco Lässig, Samuele Giuliano Piazzetta, Thomas Neuer, Léo Tafti, Guillaume Raille, Tomas Van Pottelbergh, Marek Pasić, Andrzej Skrodzki, Nicolas Huguenin, Maxime Dumonal, Jan Kościsz, Dennis Bader, Frédéric Gusset, Mounir Benheddi, Camila Williamson, Michal Kosinski, Matej Petrik, and Gaël Grosch. 2022. Darts: user-friendly modern machine learning for time series. *J. Mach. Learn. Res.* 23, 1, Article 124 (Jan. 2022), 6 pages.
- [19] Geoffrey E. Hinton. 1989. Connectionist learning procedures. *Artificial Intelligence* 40, 1 (1989), 185–234. doi:10.1016/0004-3702(89)90049-0
- [20] Hassan Ismail Fawaz, Benjamin Lucas, Germain Forestier, Charlotte Pelletier, Daniel F. Schmidt, Jonathan Weber, Geoffrey I. Webb, Lhassane Idoumghar, Pierre-Alain Muller, and François Petitjean. 2020. InceptionTime: Finding AlexNet for Time Series Classification. *Data Mining and Knowledge Discovery* (2020).
- [21] Ming Jin, Huan Yee Koh, Qingsong Wen, Daniele Zambon, Cesare Alippi, Geoffrey I. Webb, Irwin King, and Shirui Pan. 2024. A Survey on Graph Neural Networks for Time Series: Forecasting, Classification, Imputation, and Anomaly Detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46, 12 (2024), 10466–10485. doi:10.1109/TPAMI.2024.3443141
- [22] Taesung Kim, Jinhee Kim, Yunwon Tae, Cheonbok Park, Jang-Ho Choi, and Jaegul Choo. 2021. Reversible Instance Normalization for Accurate Time-Series Forecasting against Distribution Shift. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=cGDakQo1C0p>
- [23] Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. 2018. Modeling Long- and Short-Term Temporal Patterns with Deep Neural Networks. In *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval* (Ann Arbor, MI, USA) (SIGIR '18). Association for Computing Machinery, New York, NY, USA, 95–104. doi:10.1145/3209978.3210006
- [24] Yan Li, Xinjiang Lu, Yaqing Wang, and Dejing Dou. 2022. Generative Time Series Forecasting with Diffusion, Denoise, and Disentanglement. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 23009–23022. https://proceedings.neurips.cc/paper_files/paper/2022/file/91a85f3bf8f570e6be52b335b5ab017a-Paper-Conference.pdf
- [25] Yaguang Li, Rose Yu, Cyrus Shahabi, and Yan Liu. 2018. Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting. In *International Conference on Learning Representations (ICLR '18)*.
- [26] Xu Liu, Junfeng Hu, Yuan Li, Shizhe Diao, Yuxuan Liang, Bryan Hooi, and Roger Zimmermann. 2024. UniTime: A Language-Empowered Unified Model for Cross-Domain Time Series Forecasting. In *The Web Conference 2024*. <https://openreview.net/forum?id=P6sKyx2xAB>
- [27] Yong Liu, Tengge Hu, Haoran Zhang, Haixu Wu, Shiyu Wang, Lintao Ma, and Mingsheng Long. 2024. iTransformer: Inverted Transformers Are Effective for Time Series Forecasting. In *The Twelfth International Conference on Learning Representations*. <https://openreview.net/forum?id=jePFAI8fah>
- [28] Yong Liu, Haixu Wu, Jianmin Wang, and Mingsheng Long. 2024. Non-stationary transformers: exploring the stationarity in time series forecasting. In *Proceedings of the 36th International Conference on Neural Information Processing Systems* (New Orleans, LA, USA) (NIPS '22). Curran Associates Inc., Red Hook, NY, USA, Article 718, 13 pages.
- [29] Zhiding Liu, Mingyue Cheng, Zhi Li, Zhenya Huang, Qi Liu, Yanhu Xie, and Enhong Chen. 2023. Adaptive Normalization for Non-stationary Time Series Forecasting: A Temporal Slice Perspective. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 14273–14292.
- [30] Michael W. McCracken and Serena Ng. 2016. FRED-MD: A Monthly Database for Macroeconomic Research. *Journal of Business & Economic Statistics* 34, 4 (2016), 574–589. arXiv:https://doi.org/10.1080/07350015.2015.1086655 doi:10.1080/07350015.2015.1086655
- [31] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *International Conference on Learning Representations*.
- [32] George Panagopoulos, Giannis Nikolentzos, and Michalis Vazirgiannis. 2020. Transfer Graph Neural Networks for Pandemic Forecasting. arXiv (2020). <https://europepmc.org/article/PPR/PPR272396>
- [33] R. Poyatos, V. Granda, V. Flo, M. A. Adams, B. Adorján, D. Agudé, and et. al. 2021. Global transpiration data from sap flow measurements: the SAPFLUXNET database. *Earth System Science Data* 13, 6 (2021), 2607–2649. doi:10.5194/essd-13-2607-2021
- [34] Xiangfei Qiu, Jilin Hu, Lekui Zhou, Xingjian Wu, Junyang Du, Buang Zhang, Chenjuan Guo, Aoying Zhou, Christian S. Jensen, Zhenli Sheng, and Bin Yang. 2024. TFB: Towards Comprehensive and Fair Benchmarking of Time Series Forecasting Methods. *Proc. VLDB Endow.* 17, 9 (Aug. 2024), 2363–2377. doi:10.14778/3665844.3665863
- [35] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer. *Journal of Machine Learning Research* 21, 140 (2020), 1–67. <http://jmlr.org/papers/v21/20-074.html>
- [36] Pablo Romeu-Gualart and Francisco Zamora-Martinez. 2014. SML2010. UCI Machine Learning Repository. DOI:https://doi.org/10.24432/C5RS3S.
- [37] David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. 2020. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting* 36, 3 (2020), 1181–1191.
- [38] Chao Song, Youfang Lin, Shengnan Guo, and Huaiyu Wan. 2020. Spatial-Temporal Synchronous Graph Convolutional Networks: A New Framework for Spatial-Temporal Network Data Forecasting. *Proceedings of the AAAI Conference on Artificial Intelligence* 34, 01 (Apr. 2020), 914–921. doi:10.1609/aaai.v34i01.5438
- [39] Emmanouil Sylligardos, Paul Boniol, John Paparrizos, Panos Trahanias, and Themis Palpanas. 2023. Choose Wisely: An Extensive Evaluation of Model Selection for Anomaly Detection in Time Series. *Proc. VLDB Endow.* 16, 11 (July 2023), 3418–3432. doi:10.14778/3611479.3611536

- [40] Hiro Y. Toda and Peter C. B. Phillips. 1994. Vector autoregression and causality: a theoretical overview and simulation study. *Econometric Reviews* 13, 2 (1994), 259–285. doi:10.1080/07474939408800286
- [41] Artur Trindade. 2015. ElectricityLoadDiagrams20112014. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C58C86>.
- [42] Hristos Tyrallis and Georgia Papacharalampous. 2017. Variable selection in time series forecasting using random forests. *Algorithms* 10, 4 (2017), 114.
- [43] J. van der Hooft, S. Petrangeli, T. Wauters, R. Huysegems, P. Rondao Alfai, T. Bostoen, and F. De Turck. 2016. HTTP/2-Based Adaptive Streaming of HEVC Video Over 4G/LTE Networks. *IEEE Communications Letters* 20, 11 (2016), 2177–2180.
- [44] Saverio Vito. 2008. Air Quality. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C59K5F>.
- [45] Huiqiang Wang, Jian Peng, Feihu Huang, Jince Wang, Junhui Chen, and Yifei Xiao. 2023. MICN: Multi-scale Local and Global Context Modeling for Long-term Series Forecasting. (2023).
- [46] Yuxuan Wang, Haixu Wu, Jiaxiang Dong, Guo Qin, Haoran Zhang, Yong Liu, Yunzhong Qiu, Jianmin Wang, and Mingsheng Long. 2024. TimeXer: Empowering Transformers for Time Series Forecasting with Exogenous Variables. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*. <https://openreview.net/forum?id=INAEUQ04IT>
- [47] Zhiguang Wang, Weizhong Yan, and Tim Oates. 2017. Time series classification from scratch with deep neural networks: A strong baseline. In *2017 International Joint Conference on Neural Networks (IJCNN)*. 1578–1585. doi:10.1109/IJCNN.2017.7966039
- [48] Gerald Woo, Chenghao Liu, Akshat Kumar, Caiming Xiong, Silvio Savarese, and Doyen Sahoo. 2024. Unified Training of Universal Time Series Forecasting Transformers. In *Forty-first International Conference on Machine Learning*.
- [49] Haixu Wu, Tengge Hu, Yong Liu, Hang Zhou, Jianmin Wang, and Mingsheng Long. 2023. TimesNet: Temporal 2D-Variation Modeling for General Time Series Analysis. In *International Conference on Learning Representations*.
- [50] Haixu Wu, Jiehui Xu, Jianmin Wang, and Mingsheng Long. 2021. Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting. In *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (Eds.), Vol. 34. Curran Associates, Inc., 22419–22430. https://proceedings.neurips.cc/paper_files/paper/2021/file/bcc0d400288793e8bdcd7c19a8ac0c2b-Paper.pdf
- [51] Zonghan Wu, Shirui Pan, Guodong Long, Jing Jiang, Xiaoju Chang, and Chengqi Zhang. 2020. Connecting the Dots: Multivariate Time Series Forecasting with Graph Neural Networks. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (Virtual Event, CA, USA) (KDD '20)*. Association for Computing Machinery, New York, NY, USA, 753–763.
- [52] Yuanxiang Ying, Juanyong Duan, Chunlei Wang, Yujing Wang, Congrui Huang, and Bixiong Xu. 2020. Automated Model Selection for Time-Series Anomaly Detection. arXiv:2009.04395 [cs.LG] <https://arxiv.org/abs/2009.04395>
- [53] Bing Yu, Haoteng Yin, and Zhanxing Zhu. 2018. Spatio-temporal graph convolutional networks: a deep learning framework for traffic forecasting. In *Proceedings of the 27th International Joint Conference on Artificial Intelligence (Stockholm, Sweden) (IJCAI'18)*. AAAI Press, 3634–3640.
- [54] Chengqing Yu, Fei Wang, Zezhi Shao, Tangwen Qian, Zhao Zhang, Wei Wei, and Yongjun Xu. 2024. GinAR: An End-To-End Multivariate Time Series Forecasting Model Suitable for Variable Missing. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (Barcelona, Spain) (KDD '24)*. Association for Computing Machinery, New York, NY, USA, 3989–4000. doi:10.1145/3637528.3672055
- [55] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. 2023. Are transformers effective for time series forecasting?. In *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence (AAAI'23/IAAI'23/EAAI'23)*. AAAI Press, Article 1248, 8 pages. doi:10.1609/aaai.v37i9.26317
- [56] Ailing Zeng, Muxi Chen, Lei Zhang, and Qiang Xu. 2023. Are Transformers Effective for Time Series Forecasting? *Proceedings of the AAAI Conference on Artificial Intelligence*.
- [57] Shuyi Zhang, Bin Guo, Anlan Dong, Jing He, Ziping Xu, and Song Chen. 2017. Cautionary tales on air-quality improvement in Beijing. *Proceedings of the Royal Society A: Mathematical, Physical and Engineering Science* 473 (09 2017), 20170457. doi:10.1098/rspa.2017.0457
- [58] Yunhao Zhang and Junchi Yan. 2023. Crossformer: Transformer Utilizing Cross-Dimension Dependency for Multivariate Time Series Forecasting. In *International Conference on Learning Representations*.
- [59] Yue Zhao, Ryan A. Rossi, and Leman Akoglu. 2021. Automatic unsupervised outlier model selection. In *Proceedings of the 35th International Conference on Neural Information Processing Systems (NIPS '21)*. Curran Associates Inc., Article 343, 14 pages.
- [60] Haoyi Zhou, Shanghang Zhang, Jieqi Peng, Shuai Zhang, Jianxin Li, Hui Xiong, and Wancai Zhang. 2021. Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting. In *The Thirty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2021, Virtual Conference*, Vol. 35. AAAI Press, 11106–11115.
- [61] Tian Zhou, Ziqing Ma, Xue Wang, Qingsong Wen, Liang Sun, Tao Yao, Wotao Yin, and Rong Jin. 2024. FiLM: frequency improved legendre memory model for long-term time series forecasting. In *Proceedings of the 36th International Conference on Neural Information Processing Systems (New Orleans, LA, USA) (NIPS '22)*. Curran Associates Inc., Red Hook, NY, USA, Article 921, 14 pages.
- [62] Tian Zhou, Ziqing Ma, Qingsong Wen, Xue Wang, Liang Sun, and Rong Jin. 2022. FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting. In *Proc. 39th International Conference on Machine Learning (ICML 2022)* (Baltimore, Maryland).
- [63] Tian Zhou, Peisong Niu, Xue Wang, Liang Sun, and Rong Jin. 2023. One Fits All: Power General Time Series Analysis by Pretrained LM. In *NeurIPS*.

A Related Work

Large time series model. MOMENT [16] is the first time series foundation model. [48] construct a universal time series forecasting Transformer (Moirai). Prior to this, most large time series models were built on top of large language models (LLMs). Chronos-T5 [1] is a T5 LLM-based large time series model for forecasting tasks. GPT4TS [63] uses the pre-trained GPT-2 to perform time series tasks. TEMPO [8] also utilizes GPT as the backbone and is a prompt-based generative pre-training transformer for time series forecasting tasks. Compared to the MTSF method, large time series models have more generalized time series processing abilities and can be applied to various domains. However, training such models can be time and resource-consuming. Besides, due to the need to consider multiple domain data, the forecasting performances of large models are generally not as effective as those of specifically designed MTSF methods in many cases.

B Datasets

Table 1 gives the details of 41 multivariate time series datasets used in this work.

C MTSF Methods

We selected 17 MTSF methods as candidate MTSF methods for model selection and compared AutoMTSF with these methods in our experiments. The classifications of 17 MTSF methods are shown in Table 2. These methods are listed below:

TimeXer [46]: utilizes patch-level and variate-level representations for endogenous and exogenous variables, respectively, with an endogenous global token serving as an intermediary.

iTransformer [27]: The temporal embeddings of each time step are encoded as variable tokens, and a multi-head attention mechanism is employed to capture inter-variable dependencies.

Algorithm 1: Calculating Seasonality Strength

Input: Time series X

Output: Seasonality strength S_{sea} of X

- 1 $\bar{X} \leftarrow \text{mean}(X)$;
 - 2 $A \leftarrow |FFT(X - \bar{X})|$;
 - 3 $A_{pos} \leftarrow A[1 : \lfloor \frac{\text{len}(A)}{2} \rfloor]$;
 - 4 $id \leftarrow \arg \max(A_{pos}[1 :]) + 1$;
 - 5 $S_{sea} \leftarrow \frac{A_{pos}[id]}{\sum A_{pos}}$;
 - 6 return $S_{sea} \leftarrow \min(1.0, 5 \times S_{sea})$;
-

Table 1: Details of 41 multivariate time series datasets.

Dataset	Domain	Fre	Len	Channel
Bus_A [43]	Network	1 second	6893	18
Bus_B [43]	Network	1 second	3890	18
Car_A [43]	Network	1 second	116870	18
Car_B [43]	Network	1 second	17439	18
Pedestrian_A [43]	Network	1 second	29118	18
Pedestrian_B [43]	Network	1 second	4511	18
Static_A [43]	Network	1 second	12963	18
Static_B [43]	Network	1 second	2298	18
Train_A [43]	Network	1 second	32681	18
Train_B [43]	Network	1 second	6295	18
AQShunyi [57]	Environment	1 hour	35064	11
AQWan [57]	Environment	1 hour	35064	11
AirQuality [44]	Environment	1 hour	9358	13
IntelLab [5]	Environment	1 mins	2313153	6
Szeged [7]	Environment	1 hour	96454	8
Weather [50]	Environment	10 mins	52696	21
SML [36]	Environment	15 mins	2764	24
SML2 [36]	Environment	15 mins	1373	24
Electricity [41]	Electricity	1 hour	26304	321
ETTh1 [60]	Electricity	1 hour	14400	7
ETTh2 [60]	Electricity	1 hour	14400	7
ETTm1 [60]	Electricity	15 mins	57600	7
ETTm2 [60]	Electricity	15 mins	57600	7
Traffic [50]	Transportation	1 hour	17544	862
METR-LA [38]	Transportation	5 mins	34272	207
PEMS04 [38]	Transportation	5 mins	16992	206
PEMS08 [38]	Transportation	5 mins	17856	170
PEMS-BAY [25]	Transportation	5 mins	52116	81
Solar [23]	Energy	10 mins	52560	137
Wind [24]	Energy	15 mins	48673	7
WindPower [4]	Energy	24 hours	6575	8
Exchange [23]	Economic	24 hours	7588	8
FRED-MD [30]	Economic	1 month	728	107
NASDAQ [13]	Stock	24 hours	1244	5
NYSE [13]	Stock	24 hours	1243	5
CzeLan [33]	Nature	30 mins	19934	11
ZafNoo [33]	Nature	30 mins	19225	11
Covid-19 [32]	Health	24 hours	1392	948
ILI [50]	Health	1 week	966	7
NN5 [3]	Bank	24 hours	791	111
Wike2000 [15]	Web	24 hours	792	2000

Furthermore, a feed-forward network is applied to each variable token to learn non-linear representations.

Crossformer [58]: divides MTS data into segments along each dimension and embeds these segments into feature vectors. A two-stage self-attention layer is used to efficiently capture the dependencies between time and different dimensions and a hierarchical encoder-decoder is used to exploit the information at different scales for prediction.

DLinear [55]: combines a decomposition scheme from Autoformer and FEDformer with linear layers to predict time series. It models trend and seasonal components separately and sums their features for enhanced performance in trend-rich datasets.

FEDformer [62]: leverages seasonal trend decomposition with Transformers for MTSF and uses frequency insights to improve efficiency and accuracy.

FiLM [61]: applies Legendre polynomial projections to approximate historical information, uses Fourier projections to remove noise, and adds low-rank approximations to speed up computation.

Informer [60]: leverages ProbSparse self-attention for efficiency, self-attention distilling for handling long inputs, and a generative decoder for rapid prediction.

Linear [18]: makes predictions based on linear relationships between time series data.

MICN [45]: first extracts the local features of the time series using a multi-scale hybrid decomposition, and then extracts the correlation between all the local features to obtain the global features. Instead of Attention with high complexity, the newly proposed Isometric Convolution module is used in modelling the global features.

NLinear [55]: the input is first subtracted from the last value of the sequence, then the input is passed through a linear layer and the subtraction is added back before the final prediction is made. This design avoids large shifts in the curve.

Nonstationary Transformer [28]: is specially designed for non-stationary time-series prediction problems, and contain a pair of complementary Series Stationarisation and De-stationary Attention modules, which can be widely applied to the Transformer and its variants to consistently improve its prediction effect on non-smooth time series data.

PatchTST [31]: is a Transformer-based MTSF model that splits data into sub-sequence patches with a channel-independent design, which effectively reduces the computational cost while improving the long-term forecasting accuracy.

RNN [22]: The RNN remembers the historical information and applies it to the computation of the current output, i.e., the nodes between the hidden layers are connected and the inputs to the hidden layers include not only the outputs of the input layers but also the outputs of the hidden layers at the previous moment.

TCN [2]: uses convolutional layers to extract local features in a time series and uses residual blocks to capture long-term dependencies in the series.

TimesNet [49]: converts one-dimensional time series into two-dimensional tensors, captures intra- and inter-period variations, and extracts complex temporal patterns using the TimesBlock with an initial block.

Triformer [11]: uses a light-weight method to enable distinct sets of model parameters for different time series to enhance accuracy without compromising efficiency and memory usage.

VAR [40]: assumes that the current value of each variable is correlated with the values of all the variables in the past period. Thus, it predicts future values of the variables by building a regression model that includes all the variables.

D Time Series Classifier

We compare the classification accuracy of AutoMTSF with ConvNet, ResNet, and InceptionTime. The parameters used for these classifiers are listed in Table 3-5. The in_channel of ResNet and InceptionTime are equal to the sum of the horizon and sequence length.

Table 2: Classifications of 17 MTSF methods

Method	Statistical learning method	Machine learning method	Deep learning method				Model-agnostic-based
			MLP-based	RNN-based	CNN-based	Transformer-based	
VAR	✓						
Linear		✓					
NLinear			✓				
Dinear			✓				
RNN				✓			
MICN					✓		
TimesNet					✓		
TCN					✓		
PatchTST						✓	
Crossformer						✓	
FEDformer						✓	
Informer						✓	
Triformer						✓	
TimeXer						✓	
iTransformer						✓	
FiLM							✓

Table 3: Parameter setting of ConvNet.

Parameter	Value
num_blocks	5
kernel_size	3
padding	1
original_dim	1
num_classes	15

Table 4: Parameter setting of ResNet.

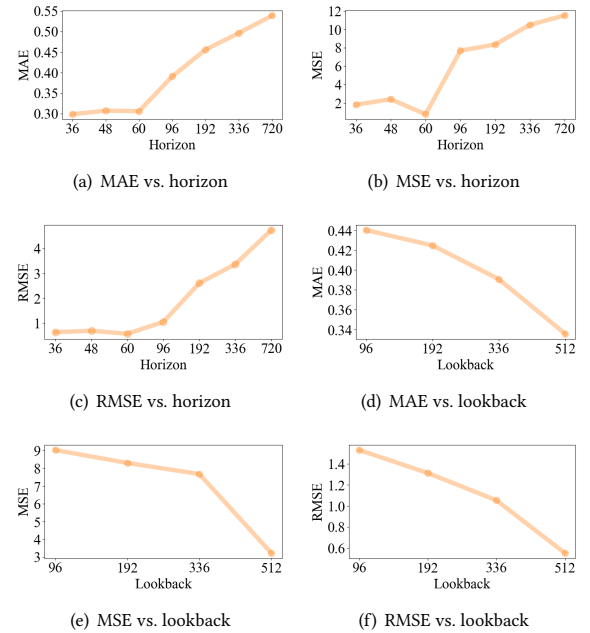
Parameter	Value
mid_channels	64
num_layers	3
num_pred_classes	15

Table 5: Parameter setting of InceptionTime.

Parameter	Value
num_blocks	5
out_channels	64
bottleneck_channels	32
kernel_sizes	[10,20,40,80,160]
use_residuals	true
num_pred_classes	15

E Parameter Sensitivity Analysis

Horizon. Figure 1(a)-1(c) illustrates average forecasting results of AutoMTSF over 41 datasets as horizon value changes. It is generally believed that forecasting accuracy typically declines with increasing horizon values. However, AutoMTSF's performance improves when the horizon increases from 48 to 60. This is because AutoMTSF performs feature extraction and classification based on

**Figure 1: Forecasting accuracy vs. horizon/lookback.**

data consisting of historical lookback length and future horizon length. Longer data enhances learning and MTSF method selection. However, when the horizon significantly increases ($H > 60$), forecasting difficulty outweighs data length benefits, leading to decreased accuracy.

Lookback. The effect of lookback on AutoMTSF's forecasting accuracy with horizon $H = 96$ is shown in Figure 1(d)-1(f). As expected, the larger the lookback, i.e., the more historical data that can be referred to, the better the forecasting performance of AutoMTSF.